

Urubu do Pix: pagamentos instantâneos distribuídos

Proposta de tema · Projeto 01 · Engenharia de Sistemas Distribuídos 2026.2

Equipe: Deivison Costa (@Deivison-Costaa) · Giancarlo Silveira Cavalcante (@GianBala) · Herlan Alef de Lima Nascimento (@HerlanLima) · Integrante 4 · Integrante 5

Repositório: github.com/Deivison-Costaa/urubu-do-pix

1. Problema

O "urubu do Pix" é o golpe que promete multiplicar o dinheiro enviado. Este projeto garante o contrário: nenhum centavo é criado ou perdido. Um pagamento instantâneo atravessa pelo menos três participantes independentes (banco pagador, liquidante central e banco recebedor), cada um com o próprio banco de dados. Ele precisa terminar em segundos e manter a soma dos saldos intacta mesmo com quedas de serviço, timeouts, partições de rede e mensagens repetidas. Como não existe transação distribuída (2PC) entre instituições, essa garantia tem de vir da arquitetura. O sistema implementa uma versão simplificada do ecossistema Pix: N bancos simulados (PSPs), o SPI (liquidação entre contas de reserva), o DICT (diretório de chaves Pix) e um serviço antifraude.

2. Solução

O PSP pagador consulta a chave no DICT, pede o score ao antifraude, debita o cliente e publica a ordem. O SPI liquida de forma atômica entre as contas de reserva, e o PSP recebedor credita e confirma. Se o recebedor recusar ou não responder no prazo, o SPI dispara a devolução ao pagador. O EndToEndId de cada pagamento é a chave de idempotência em todas as etapas. Um auditor verifica os invariantes ao fim de cada teste.

3. Padrões arquiteturais (Seção 5)

Padrão	Onde será aplicado	Área
SAGA (orquestração) + Transactional Outbox	ciclo de vida do pagamento entre PSPs e SPI	Confiabilidade
Retry + DLQ	entrega de mensagens entre serviços	Confiabilidade
Circuit Breaker + Request Hedging	chamadas ao antifraude (consulta idempotente)	Confiabilidade, Desempenho
API Gateway + Rate Limit	entrada externa; bloqueio de varredura de chaves no DICT	Segurança, Desempenho
Cache-Aside	consultas de chaves no DICT	Desempenho
Database-per-service; Single vs Multi-tenant	um banco por serviço; o mesmo código de PSP implantado por banco	Escalabilidade
Zero Trust: mTLS + OAuth 2.0/OIDC/JWT	serviço a serviço e cliente a API	Segurança
Canary Release	nova versão do antifraude com rollback automático	Deployment

O projeto cobre as cinco áreas técnicas (o mínimo exigido é duas).

4. Tecnologias previstas

Go · PostgreSQL (um por serviço) · Redpanda (API Kafka) · Redis · Envoy + OPA · Keycloak · mTLS com CA própria (evolução: SPIFFE/SPIRE) · OpenTelemetry, Prometheus, Grafana e Jaeger · Docker Compose e Kubernetes (kind) com Argo Rollouts · GitHub Actions com SAST, Trivy, SBOM (Syft) e Cosign · k6, Toxiproxy e Testcontainers. As escolhas serão justificadas em ADRs no Projeto 02.

5. Critérios de sucesso mensuráveis

#	Critério	Meta	Como medir
C1	Conservação	soma de todos os saldos idêntica antes e depois de cada cenário, inclusive de caos	auditor de invariantes
C2	Idempotência	0 liquidações duplicadas em 10.000 reenvios do mesmo EndToEndId	teste de integração
C3	Desempenho	p99 fim a fim ≤ 2 s a 300 pagamentos/s durante 10 min	k6 + Prometheus
C4	Resiliência	rede SPI ↔ PSP cortada sob carga: 100% dos pagamentos afetados em estado final (liquidado ou devolvido) até 30 s após a reconexão	Toxiproxy + auditor
C5	Degradação	antifraude fora do ar: circuit breaker abre em ≤ 5 s e pagamentos abaixo do limite configurado seguem aprovados	teste de resiliência
C6	Segurança	100% das chamadas sem token ou certificado válido recusadas; varredura do DICT bloqueada acima do limite	testes negativos
C7	Deployment	canary do antifraude volta sozinho à versão anterior se a taxa de erro passar de 1%	Argo Rollouts

6. Plano de testes

Testes unitários e de integração (Testcontainers) por serviço; carga com k6; resiliência com Toxiproxy e queda de contêineres; testes negativos de segurança. Todo cenário termina com o auditor de invariantes, que valida C1, C2 e C4 automaticamente. As medições de desempenho serão feitas numa máquina de referência descrita no README.